

PANDAS FOR DATA PROCESSING

PART 1 — WHAT IS PANDAS REALLY?

Formal Definition

Pandas is a high-level Python library built on NumPy that provides fast, flexible, and expressive data structures for structured data manipulation and analysis.

Why Data Engineers Use Pandas

Pandas is used for:

- Batch data cleaning
- Data validation before warehouse load
- Feature engineering
- Data exploration
- Pre-processing for ML
- Data transformation prototyping before Spark implementation

It is NOT:

- A distributed system
- Designed for TB-scale data
- Meant to replace Spark

It is:

- In-memory
- Column-oriented
- Highly optimized for medium-sized datasets

Architecture Overview

Pandas is built on:

- NumPy arrays (C-backed, optimized)
- Columnar memory storage

Each DataFrame column is stored as a separate contiguous block of memory.

This makes:

- Column operations fast
- Vectorized operations efficient
- Row-wise operations slower

This is why we avoid loops.

PART 2 — DATA STRUCTURES

SERIES

Definition

A Series is a one-dimensional labeled array capable of holding any data type.

Think of it as:

- A column in a table
- A single vector of data
- A mapping between index and values

Example

```
import pandas as pd
```

```
s = pd.Series([100, 200, 300])
```

Internally:

- Values stored in NumPy array
- Index stored separately

Index Concept

Every Series has an index.

```
s = pd.Series([10,20,30], index=["a","b","c"])
```

Now:

```
s["a"]
```

Index allows label-based access.

Data Types in Series

```
s.dtype
```

Common types:

- int64
- float64
- object (string)
- bool
- datetime64

Data type matters for performance and memory usage.

DATAFRAME

Definition

A DataFrame is a two-dimensional labeled data structure with columns of potentially different data types.

Think of it as:

- SQL table
- Excel sheet
- Structured dataset in memory

Internal Layout

A DataFrame is a collection of Series objects aligned by index.

Example:

```
data = {  
    "name": ["Alice", "Bob"],  
    "age": [25, 30]  
}
```

```
df = pd.DataFrame(data)
```

Internally:

- Column "name" → Series
- Column "age" → Series

This is column-oriented storage.

Why Column-Oriented Matters

Column operations are:

- Vectorized
- Faster
- Memory efficient

Row operations are slower.

PART 3 — LOADING DATA

read_csv

```
df = pd.read_csv("data.csv")
```

Important parameters:

```
df = pd.read_csv(  
    "data.csv",  
    sep=";",  
    header=0,  
    dtype={"age": "int32"},  
    parse_dates=["created_at"],  
    na_values=["NA", ""]  
)
```

- sep → delimiter
- dtype → enforce schema
- parse_dates → convert automatically
- na_values → treat as missing

1. sep (The Separator)

By default, `read_csv` looks for commas. However, data often comes in different formats. `sep` tells Pandas exactly which character marks the end of one column and the start of the next.

- **Common uses:** `sep=' ; '`, `sep='\t'` (Tab-separated), or `sep='|'`.
- **Pro Tip:** If your file has inconsistent spacing, you can use `sep='\s+'` to treat any amount of whitespace as a single delimiter.

2. dtype (Data Type Enforcement)

Pandas tries to guess your data types, but it's not psychic. If you have a column of "Employee IDs" that look like numbers (e.g., 00504), Pandas will turn them into integers and delete the leading zeros. `dtype` stops this.

- **How it looks:** `dtype={'EmployeeID': str, 'Price': float}`.
- **Why use it?** It saves memory (by using smaller types like `int32`) and ensures data integrity.

3. parse_dates (The Time Traveler)

Normally, Pandas imports dates as plain text (strings). This means you can't easily calculate the "number of days between orders." `parse_dates` tells Pandas to identify these columns and convert them into actual `datetime64` objects.

- **Standard use:** `parse_dates=['OrderDate', 'ShipDate']`.
- **The benefit:** Once parsed, you can use `.dt` accessors (like `df['OrderDate'].dt.year`) to group data by month or day.

4. na_values (The Cleanup Crew)

Sometimes "missing data" isn't just an empty cell. Different systems export "N/A" in weird ways, like `?`, `999`, or `NOT_AVAILABLE`. If you don't tell Pandas these are "missing," it will treat them as actual data (which breaks your math).

- **Standard use:** `na_values=['?', 'NONE', -999]`.
- **Result:** All those values become `NaN` (Not a Number), allowing you to use functions like `df.dropna()` or `df.fillna()`.

Schema enforcement is a Data Engineering best practice.

Memory Optimization

Large datasets:

```
df = pd.read_csv("data.csv", dtype={"age": "int16"})
```

Using smaller integer types reduces memory.

By default, Pandas usually assigns `int64` to whole numbers. Here is why your specific line of code is a pro-level tweak:

1. Memory Efficiency

The number 16 in `int16` refers to the number of bits.

- **int64:** Uses 64 bits per value.
- **int16:** Uses only 16 bits per value.

If you have a dataset with 1 million rows, switching from the default to `int16` reduces the memory footprint of that column by **75%**. This can be the difference between your script running smoothly or crashing your computer.

2. The "Range" Reality Check

When you enforce a schema, you have to ensure the data fits in the "bucket" you built.

- **int16** can hold numbers from **-32,768 to 32,767**.
- Since human ages rarely exceed 120, `int16` is more than enough.

Warning: If your CSV contains a value like 33000 or a decimal like 25.5 in the "age" column, this line of code will trigger a **ValueError** or a **TypeError** because it doesn't fit the `int16` mold.

3. When to use str instead?

As mentioned earlier, dtype is most critical for **ID numbers** or **Zip Codes**.

Python

```
# If you don't do this, Zip Code 00210 becomes 210
df = pd.read_csv("data.csv", dtype={"zip_code": str})
```

Summary of the "Size" options:

Type	Memory	Range
int8	8-bit	-128 to 127
int16	16-bit	-32,768 to 32,767
int32	32-bit	~ -2 billion to 2 billion
int64	64-bit	Effectively infinite for most data

PART 4 — DATA INSPECTION

Structural Inspection

```
df.shape
df.columns
df.dtypes
df.info()
```

- shape → rows and columns
- dtypes → schema validation
- info() → memory usage

Statistical Inspection

```
df.describe()
```

Shows:

- mean
- std
- min
- max
- percentiles

Used to detect anomalies.

Exercise

Load dataset and answer:

- How many rows?
- How many missing values?
- What is memory usage?

PART 5 — FILTERING

Filtering is equivalent to SQL WHERE.

Boolean Masking

```
df[df["salary"] > 5000]
```

Internally:

- Condition creates boolean Series
- True values retained
- False dropped

Multiple Conditions

```
df[(df["age"] > 25) & (df["salary"] > 5000)]
```

Important:

Use &, not and.

isin()

```
df[df["region"].isin(["North", "South"])]
```

Equivalent to SQL IN clause.

Query Method

```
df.query("age > 25 and salary > 5000")
```

More readable.

Exercise

- Filter rows where region is North.
- Filter rows where salary between 5000 and 8000.
- Filter rows where name starts with "A".

PART 6 — HANDLING MISSING DATA

Detect Missing

```
df.isnull()  
df.isnull().sum()
```

Drop Missing

```
df.dropna()
```

Drop specific column:

```
df.dropna(subset=["salary"])
```

Fill Missing

```
df.fillna(0)  
df["salary"].fillna(df["salary"].mean())
```

Engineering Decision

Should you:

- Drop row?
- Replace with default?
- Impute mean?
- Flag record?

Depends on business rule.

PART 7 — TRANSFORMATIONS

Creating New Columns

```
df["bonus"] = df["salary"] * 0.10
```

Vectorized — fast.

apply()

```
df["category"] = df["salary"].apply(lambda x: "High" if x > 6000 else "Low")
```

But apply is slower than vectorized operations.

Prefer:

```
df["category"] = "Low"  
df.loc[df["salary"] > 6000, "category"] = "High"
```

Sorting

```
df.sort_values("salary", ascending=False)
```

Exercise

Create:

- tax column (15%)
- net_salary
- flag high earners
- sort by net_salary descending

PART 8 — AGGREGATIONS

Aggregation reduces data.

Column Aggregation

```
df["salary"].sum()  
df["salary"].mean()  
df["salary"].std()
```

Multiple Aggregations

```
df.agg({  
    "salary": ["sum", "mean"],  
    "age": ["min", "max"]  
})
```

Custom Aggregation

```
df["salary"].agg(lambda x: x.max() - x.min())
```

Exercise

Calculate:

- Total payroll
- Average salary
- Salary variance
- Employee count

PART 9 — GROUPBY (MOST IMPORTANT FOR DE)

Concept

GroupBy splits data into groups and applies aggregation.

Equivalent to SQL:

```
SELECT region, SUM(sales)
FROM table
GROUP BY region;
```

Example

```
df.groupby("region")["sales"].sum()
```

Multiple Aggregations

```
df.groupby("region").agg({
    "sales": ["sum", "mean"],
    "age": "max"
})
```

Reset Index

```
result = df.groupby("region")["sales"].sum().reset_index()
```

Important before exporting.

Multi-Level Grouping

```
df.groupby(["region", "product"])["sales"].sum()
```

Creates hierarchical index.

Exercise

- Total sales per region.
- Total sales per region and product.
- Top region by revenue.

PART 10 — PERFORMANCE & LIMITATIONS

Vectorization

Avoid:

```
for index, row in df.iterrows():
```

Prefer:

- Column operations
- Built-in functions

Why `iterrows()` is the "Slow Way"

When you use a for loop (like `iterrows()`), Python has to look at each row one by one. It pulls the row out, looks at the value, does the math, and puts it back.

- **The overhead:** Python is a "high-level" language, meaning every time it runs a loop cycle, it has to do a lot of background checks. Doing this 1 million times is incredibly slow.

What is Vectorization?

Vectorization sends the entire column (the "vector") to highly optimized C or Fortran code running under the hood of Pandas and NumPy. These libraries perform the operation on the **entire array at once** at the hardware level.

Comparison: The "Tax" Example

Let's say you want to calculate a 10% tax on a "Price" column.

The Slow Way (Looping)

This is "Element-wise" processing. It treats the DataFrame like a standard Python list.

Python

```
for index, row in df.iterrows():  
    df.at[index, 'Tax'] = row['Price'] * 0.1
```

The Fast Way (Vectorized)

This is "Column-wise" processing.

Python

```
df['Tax'] = df['Price'] * 0.1
```

Why the "Prefer" methods are better:

1. Column Operations

As shown above, using operators like +, -, *, or / directly on columns is vectorized. Pandas automatically "broadcasts" the operation across every row simultaneously.

2. Built-in Functions

Pandas and NumPy come with optimized functions for almost everything.

- **Instead of looping to find a string:** Use `df['Name'].str.upper()`
- **Instead of looping to find a mean:** Use `df['Price'].mean()`
- **Instead of looping for logic:** Use `np.where(df['Price'] > 100, 'Expensive', 'Cheap')`

The Performance Gap

Method	Speed (approx.)	Why?
<code>iterrows()</code>	1x (Baseline)	High Python overhead per row.
<code>.apply()</code>	2x - 5x faster	Better than a loop, but still acts row-by-row.
Vectorized	100x - 1000x faster	Moves the work to optimized C code.

Memory Limits

Pandas works in memory.

If dataset > RAM:

- It crashes.

Solutions:

- Chunk processing
- Use Spark

Chunk Example

```
for chunk in pd.read_csv("large.csv", chunksize=10000):  
    process(chunk)
```

FINAL LAB — BUILD SMALL ETL

Input: sales_data.csv

Steps:

1. Load data.
2. Validate schema.
3. Clean missing values.
4. Remove invalid records.
5. Add computed columns.
6. Aggregate by region.
7. Export clean data.
8. Export summary.

employee_data.csv

Exercises:

- Filter salary > 6000
- Replace missing salary
- Handle invalid salary ("abc")
- Parse joining_date
- Group by department
- Calculate total payroll
- Add tax column
- Identify high earners

sales_data.csv

Exercises:

- Convert sales_amount to numeric
- Remove invalid rows
- Fill missing values
- Group by region
- Group by product
- Group by category

- Calculate revenue per region
- Multi-level groupby (region + category)
- Sort by highest revenue

transactions_data.csv

Exercises:

- Remove duplicate transactions
- Handle invalid amount
- Filter only Completed transactions
- Total revenue by region
- Identify top customer
- Count failed transactions